Respuestas De La Continuación De Los Tps

Pregunta 12 — .query()

¿El resultado de .query() es idéntico al de su filtro_avanzado? ¿Por qué?

Sí, .equals() devolvió True: las 174 filas resultantes son exactamente las mismas con ambos métodos. Es porque .query() no es una herramienta distinta a lo que ya veníamos usando, es solo otra sintaxis para expresar la misma condición booleana (nb_of_Cores >= 4 and Vertical_Segment == 'Desktop'); por dentro pandas arma el mismo filtro.

¿Cuál de las dos formas les parece más clara para leer?

Para dos condiciones simples ambas son igual de claras, pero .query() se lee más parecido a una frase natural ('donde el precio sea mayor a tal cosa y la categoría sea tal otra'), sin tanto corchete ni ampersand. La ventaja de .query() se nota más cuando el filtro tiene 3, 4 o más condiciones encadenadas: ahí la versión con corchetes y & se vuelve difícil de leer, mientras que .query() sigue pareciendo una oración.

¿Qué ventaja tiene usar @ en lugar de escribir el valor directamente en el texto?

Usar @valor_limite en vez de escribir el número fijo dentro del string evita tener que editar el texto del filtro cada vez que cambia el criterio: alcanza con modificar la variable en un solo lugar del código, y el filtro se actualiza solo. También reduce errores de tipeo y hace que el filtro sea reutilizable si ese valor se calcula de forma dinámica (por ejemplo, a partir de un promedio o de un input del usuario).

Pregunta 13 — .isin() y ~

¿La suma de filas incluidas + excluidas da exactamente el total? ¿Por qué siempre debería ser así?

Sí: 1388 filas incluidas (Desktop + Mobile) más 895 excluidas (Server + Embedded) dan los 2283 filas del dataset original. Esto siempre tiene que cumplirse porque .isin() y ~.isin() son condiciones complementarias: cada fila del DataFrame cumple una de las dos, nunca ninguna y nunca las dos a la vez, así que entre ambos filtros se cubre el 100% de las filas sin superposición.

¿Qué ventaja tiene .isin(['A','B','C']) frente a escribir == 'A' | == 'B' | == 'C'?

Es mucho más corto y legible, sobre todo cuando la lista de valores crece: escribir tres o cuatro condiciones con == y | encadenadas es tedioso y fácil de arruinar (olvidarse un paréntesis, repetir el nombre de columna mal). Con .isin() alcanza con pasar la lista de valores una sola vez, y además esa lista se puede armar de forma dinámica (por ejemplo, a partir de otra consulta) en lugar de tener que escribir cada condición a mano.

¿Cuándo usarían la versión con ~ en un análisis real?

El ~ es útil cuando es más fácil nombrar lo que se quiere sacar que lo que se quiere quedar. Por ejemplo, si se quiere analizar 'todos los procesadores menos los discontinuados' o 'todo el dataset excepto un par de categorías problemáticas', alcanza con listar esas pocas categorías a excluir en vez de tener que enumerar todas las demás que sí interesan.

Pregunta 14 — value_counts(), unique(), nunique()

¿Cambia la distribución de categorías entre el DataFrame completo y el filtrado? ¿Qué dice eso?

Cambia completamente. En el DataFrame completo hay cuatro categorías (Mobile 33.3%, Server 31.4%, Desktop 27.5%, Embedded 7.8%), pero en el DataFrame filtrado por filtro_avanzado queda

una sola categoría: Desktop, con el 100% de las filas. Esto pasa porque el filtro pide explícitamente Vertical_Segment == 'Desktop', así que por definición elimina cualquier otra categoría. Dice que el filtro fue efectivo para aislar exactamente el segmento que se buscaba analizar.

¿Hay alguna categoría que desapareció completamente al aplicar el filtro?

Sí, tres: Mobile, Server y Embedded desaparecen por completo del DataFrame filtrado, porque el filtro exige Vertical_Segment == 'Desktop' además de 4 núcleos o más. Solo sobrevive Desktop.

¿value_counts() y groupby().count() dan el mismo resultado? ¿Cuándo usarían cada uno?

En términos generales sí dan el mismo resultado cuando se aplican sobre la misma columna: ambos cuentan cuántas veces aparece cada valor. La diferencia es de uso: value_counts() es más directa y rápida para explorar rápido una sola columna, ya viene ordenada de mayor a menor y tiene el parámetro normalize para porcentajes. groupby().count() conviene cuando además hay que calcular otras cosas sobre cada grupo (sumas, promedios, varias columnas a la vez) porque devuelve un objeto agrupado que se puede seguir operando, no solo un conteo.

Pregunta 15 — Correlación y exportación

¿Qué par de columnas tiene la correlación más alta? ¿Tiene sentido con sus datos?

El par más correlacionado fue nb_of_Threads con nb_of_Cores. Tiene mucho sentido: los hilos (threads) de un procesador dependen directamente de la cantidad de núcleos (cada núcleo aporta uno o dos hilos según tenga o no Hyper-Threading), así que es esperable que crezcan casi siempre juntos.

¿Qué significa una correlación cercana a 1? ¿Y cercana a 0? ¿Y negativa?

Una correlación cercana a 1 significa que dos variables suben y bajan juntas de forma casi proporcional (cuando una aumenta, la otra también). Una correlación cercana a 0 significa que no hay una relación lineal clara entre esas dos variables: que una suba no permite predecir qué hace la otra. Una correlación negativa significa que se mueven en sentido contrario: cuando una variable sube, la otra tiende a bajar.

¿Por qué usamos el DataFrame completo para calcular la correlación y no el filtrado?

Porque el DataFrame filtrado solo tiene procesadores Desktop con 4+ núcleos, un subconjunto chico y muy parecido entre sí en varias características. Calcular correlaciones sobre un grupo tan homogéneo puede dar resultados poco confiables o directamente engañosos, porque hay menos variación para detectar relaciones reales. Usando el DataFrame completo se aprovecha toda la variabilidad del dataset (todos los segmentos, todas las gamas de precio y potencia), lo que da una foto más representativa de cómo se relacionan las variables entre sí.